claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-afa0642948a49ecb6.jsonl`
- SHA-256: `57bc38e4ea156a37897d73e52ccb4330b75c40e24ed9472dabbf5089e5b38638`
- Source modified: `2026-06-21T13:56:09+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

Transcript

1. user (2026-06-21T13:52:09.114Z)

```
Research the LAUNCH UX for a local web-service app aimed at NON-TECH users (a coach/parent in
India, Windows-first): options = (a) "run a command -> open browser to localhost", (b) a tiny
native launcher that starts the server + opens the browser + system tray (pywebview, Tauri
sidecar, a packaged .exe launcher), (c) Electron/Tauri wrapping the SPA pointing at the local
server, (d) one-click installer. Weigh against the owner's locked decision D10 ("self-hosted
webserver the user spins up, NOT a desktop app") -- does a friendly LAUNCHER violate D10 or
satisfy its intent? Recommend a first-run wizard (set Gemini key / pick GPU-or-cloud / point at
videos). Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer  (Python, FastAPI +
SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing),
C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-
collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included"
Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-
from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies ->
highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION
if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed
end-to-end CUJ.

KEY FACTS the orchestrator already verified:
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py),
packages=["backend","training"], console script: indexer = backend.main:main. requires-python
>=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-
url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT),
reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai
is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker
{infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter
{gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda,
torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps.
Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile +
compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-
serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry);
backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job);
backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
```

- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).
khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:13.069Z)

I'll research the launch UX for this local web-service app. Let me start by reading the key project files to ground my analysis, then do web research.

3. user (2026-06-21T13:52:15.676Z)

e68b583 fix(api): resolve a bucket-ingested video's source to a local file before stitching (#289)
43eccc1 feat(serving): M8 ? per-job cost ledger + versioned pricebook + /cost API (default-OFF) (#288)
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked) (#287)
---KHELSUTRA---
ANNOTATE_AND_TRAIN.md
DRY_RUN_CUJ.md
FRIENDLY_AND_MULTILINGUAL.md
LOCAL_APPLIANCE_ARCHITECTURE.md
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md

4. user (2026-06-21T13:52:16.583Z)

C:\Users\avidu\Projects\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md

5. user (2026-06-21T13:52:16.600Z)

docs\DESKTOP_APP_PLAN.md

6. assistant (2026-06-21T13:52:18.758Z)

Let me read the key decision docs and packaging files to ground my analysis.

7. user (2026-06-21T13:52:20.147Z)

1 # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3 > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5 bridge) · **Owner:** Avi · **Created:** 2026-06-20 · **Last updated:** 2026-06-20
4 > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
5 > **Tracking anchors:** §9 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6 > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha screens. Engine-side changes here are **proposals to coordinate** with the active decoupled/worker sessions ? not written unilaterally.
7 > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not legal/financial advice.
8

---

0. TL;DR

KhelSutra Studio turns the engine from a CLI into a **single-box, UI-powered rally indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**, drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on **one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the degenerate collapse of the same storage/worker split.

The **single most important caveat:** the engine has **zero app-layer auth and CORS `*`** `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`, and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the tenancy boundary.**

Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ? provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel** with the UI MVP, which ships on Gemini today.

1. Problem & goal

The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker {infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole loop on a single box, **honestly, behind a gate**.

Headline CUJ (owner's words): a **hosted website** where I pick an input ? **a Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload** ? it lands in the **bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host** running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**

Outcome ("good"): an operator signs in, picks a source, watches it index, curates rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host later** (native WASB) with **no UI fork**.

Read to get current: `PER_RALLY_SELECTION.md` (§3/§4/§10 contract anchors), engine `HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built), `VIDEO_LOCALITY_MODEL.md` (privacy/retention).

2. Decisions locked

#	Decision	Source / date	Implication
D1	Reverse-proxied **single origin**; engine stays `127.0.0.1:8000`	`[verified main.py:638]`; 2026-06-20	CORS `*` is moot from the browser ? the proxy is the boundary
D2	**Edge auth**, zero engine auth code	`HOSTING_PLAN.md`; 2026-06-20	Caddy basicauth OR cloudflared + Cloudflare Access
D3	MVP drives the **in-process FastAPI worker** loop, `storage=local`	`[verified job_worker.py:37, local.py:33]`; 2026-06-20	bucket worker = reserved scale path, out of MVP
D4	Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim**	PR [#11]; 2026-06-20	the console is a superset; do not redesign the picker
D5	Console lives in the khelsutra **`web/`** React scaffold	`web/README.md`; 2026-06-20	Cloudflare-Pages-style root, not engine static
D6	Compute placement is a **setting**, rendered from `GET /api/capabilities`	`[design]`; 2026-06-20	UI offers ONLY what the box can honestly do
D7	**Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to one video object	`[verified backend/storage/ref.py parse_uri]`; 2026-06-20	Drive is a **stub** today ? scoped honestly (§4.2)
D8	GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage stays the bucket	Owner 2026-06-20	compute and storage are separate planes; no provider lock-in
D9	**Observability is first-class** ? structured logs on every new path + a DoD item	Owner 2026-06-20	see §5; verified during CUJ replay

| D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |

3. Foundation ? what the engine gives us today `[verified this session unless noted]`
- **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ? `POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}` (FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT /api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}` `[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-wiring only**, no engine change.
- **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)` `[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a **separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1 to serialize) ? it is **not** already 1.
- **Storage:** `storage.backend` default = `"local"` `[verified models.py:298]`; `LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on `src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://` (AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub (emulated only)**. Credentials never in config ? boto3 default chain / `~/.aws` `[verified s3.py]`.
- **Compute seam:** `indexing.detector_impl` ? {`auto`=WSL, `native`=Linux-native, `stub`=CPU mock}; default `"auto"` `[verified models.py:129]`, resolved in `trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch `feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`; **`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified :57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ? not exercisable on the droplet or in the offline suite. Treat native as **not production-verified**.
- **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ? pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` + `--set indexing.skip_ai_handoff=true` + `storage.s3`.
- **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET /api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}` `[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
- **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md, DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run there.
- **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row `get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.

4. Design / architecture

4.1 Topology & request path
ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy / static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI `127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing site stays a **separate** systemd unit on a **separate** subdomain.

```
PUBLIC INTERNET
   ?  (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
   ?      cloudflared Tunnel + Cloudflare Access (no inbound ports)  OR  Caddy :443 +
```

```
basicauth
  ?
??????????????????????????????????????????????????????????????????????????????
? ONE BOX   firewall BLOCKS inbound 8000 + 9000/9001/9023                    ?
? (2) reverse proxy / static host (Caddy/cloudflared)                        ?
?      /, /assets/*  ? web/ React SPA      /api/*, /api/highlights/* ? :8000 ?
?            ?  ONE ORIGIN to the browser ? engine CORS '*' is irrelevant    ?
?            ?                                                               ?
? (3) ENGINE FastAPI  (host=127.0.0.1:8000, HARD-bound [main.py:638])       ?
?      control: jobs table + ThreadPoolExecutor(max_workers=1) drain         ?
?      data: uploads.py chunked upload · /api/compile · /api/highlights      ?
?            ?                                                               ?
? (4) STORAGE = LocalStorage (default)  fetch=identity · put=no-op(src==dst) ?
?            ?                                                               ?
? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet]          ?
?                   native WASB (GPU host) · auto (owner Win+WSL)            ?
??????????????????????????????????????????????????????????????????????????????
RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO Spaces)
   ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ? outputs/<vid>/ back
   (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll contract)
```

4.2 Ingest sources (the headline) ? one menu, one normalized video object
All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a `StorageRef` (scale):

Source	Path today	Status
Local upload	SPA chunks the file ? `max_part_mb` via the existing `/api/upload/init|part|complete` `[verified uploads.py]` ? server path ? `POST /api/process`. (Scale: also `StorageBackend.put` to `s3://`/`gcs://`.)	**wire client only** (endpoints exist)
Bucket URI (`s3://`, `gs://`)	**`POST /api/process` accepts the bucket URI directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283 ? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the **engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs` is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket URI 400.)*	**engine: DONE (verified); SPA ingest call: design (= B-P2)**
Google Drive (`gdrive://`)	`parse_uri` recognizes it, but the gdrive backend is an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]`	**stub ? roadmap;** MVP requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real

4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)

Placement	Runs	Needs	Real/mock	Honest caveat
CPU droplet ? gemini (MVP target)	`segmenter=gemini`, in-proc worker	`GEMINI_API_KEY`; network; **no GPU**	**REAL**	Cloud brain: uploads ?1280px chunks to Google ? **must be in consent copy**; **no per-video cost ceiling** today
CPU droplet ? stub	trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff`	nothing	**MOCK**	not real tracking ? UI must label "stub (CPU mock ? not real detection)"; demo/regression only
CPU droplet ? motion	`segmenter=motion`	nothing	REAL, low-quality	only no-key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated)
GPU host ? native WASB (in-flight)	`wasb`+`detector_impl=native` ? `NativeWasbRunner` (in-proc torch)	a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU**; `WASB_WEIGHTS`	**REAL**, on-device	partially wired (fusion builds it, trajectory refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial**
Owner Win+WSL ? auto	`wasb`+`detector_impl=auto` (WSL2+conda)	Windows+WSL2+GPU	REAL (today)	Windows-only transport; the working real-GPU path for owner verification
Remote GPU worker via bucket (reserved)	console CPU + `storage=s3`; separate GPU host runs `backend.worker infer`	bucket creds; a GPU host; a **net-new orchestration		

endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as a setting" is **design** ? selector shown **disabled** in MVP |
99
100 **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the **containerization feasibility check** above.
101
102 ### 4.4 Auth / exposure (non-negotiable invariant)
103 No engine auth, by design; the gate is external. **Invariant (all three must hold):** engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist `[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**; set `security.allowed_video_root`; rate-limit; per-user quota.
104
105 ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106 - **S1 Ingest & Process** ? the source menu (§4.2) + a segmenter picker from `/api/capabilities` ? `POST /api/process`.
107 - **S2 Progress** ? 3s poll of the free-text stage (`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-failure terminal state.
108 - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`): honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter. **Per-rally is ONE screen of the console.**
109 - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for **reload-survival**.
110 - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112 ### 4.6 Reuse map
113 **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing `NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the `HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units; `GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker bridge.
114
115 ## 5. Observability ? logging & monitoring `[required, first-class]`
116 Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously, with a full log-analysis pass across every new code path.** The engine already uses a centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new path.
117
118 - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`) **plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ? GPU host. Log it on every line.
119 - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry / result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with the same correlation id.
120 - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must **never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121 - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a ledger exists) for the engine-mode banner + alerting.
122 - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is **ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the instance.
123 - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay** (§8 Layers 1?3). Ties [[feedback-launch-quality-bar]].
124
125 ## 6. Risk table
126 | Risk | Today | Mitigation |
127 |---|---|---|

128 | No app auth + CORS `*` `[main.py:42]` | open if any of {localhost-bind, firewall, gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside |
129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes them |
130 | Retention | `output/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only before any non-owner exposure |
131 | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap; `ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-production-verified until owner byte-parity on a real GPU box |
133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist | coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD, explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted in chat or committed (cf. the **already-rotated Spaces key** exposed in chat, `DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive / `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token, revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action every time** |
138
139 ## 7. Honest limits ? what this does NOT do
140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not** containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the **console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core); **CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters + honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure; `query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ? CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`). The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug (CUJ-3), capability honesty (CUJ-4).
147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main --server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema, compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1 with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption; reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job + headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no stacking**; coverage at/above the floor; **logs inspected during replay** (§5).
150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/`cv2`-vs-`ffmpeg` behavior; a multi-GB browser upload. Reported back into §9, stated plainly as **not verified**

in the offline suite.

9. Deliverables & progress tracker ? **source of truth**

Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR ships its tests + meaningful logs (§5).

ID	Deliverable	Depends on	Gated?	Status	PR
T0	This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers	?	owner	?	(this PR)
T1	Mock engine + capability fixtures + browser-replay CUJ scripts (L1)	T0	no	?	?
T2	Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key (P1)	?	owner	?	?
T3	L3 live posture test (raw `:8000`/`:900x` refused from outside)	T2	no	?	?
T4	S1 ingest (local upload wired) + segmenter picker	T1	no	?	?
T5	S2 progress (poll + one-job + restart-failure)	T4	no	?	?
T6	S3 embed **RallyPicker** + compile + download	T4	no	?	?
T7	*(engine)* `GET /api/capabilities` + `GET /api/health` + banner	T1	coord	?	?
T8	*(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival)	?	coord	?	?
T9	S5 Settings (config+capabilities; disabled compute/storage target)	T7,T8	no	?	?
T10	Observability pass: correlation id end-to-end + structured logs + metrics (§5)	T4	no	?	?
T11	Retention sweep for `output/`+`uploads/`	?	owner	?	?
T12	Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE** (`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (= **B-P2**)	T4	coord	?	engine done; SPA pending
T13	**GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner byte-parity (P4)	branch	? owner	?	?
T14	*(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5)	T8,T12	? owner	?	?

Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP does **not** block on it (ships on Gemini).

10. Open questions ? owner / external

Blocking gates: (1) **edge-auth** ? Caddy basicauth (self-contained) vs cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention defaults** ? auto-delete originals after compile? keep proxies+intervals only? window? (privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4) **GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration endpoint + how it coordinates with the active worker sessions.

Nice-to-knows: console home (web/ vs engine static); a review-playback streaming endpoint; per-user scoping timing; containerization vs runbook.

11. Definition of done

- [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via the ingest menu.
- [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
- [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped honestly (not faked).
- [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
- [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via `/api/videos` + `/api/reels`.
- [ ] **Observability:** correlation id threaded end-to-end; structured logs at every new path; loud failures; logs inspected during CUJ replay.
- [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
- [ ] CI green (L2 contract + L4 regression); coverage ? floor; §9 tracker all

?/?-resolved.

12. References
Engine code `[verified this session]`: `backend/main.py:12-17,42,52,307-328,331-340,342-396,638`; `backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`; `backend/api/static/app.js:88,344,373`; `backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`; `backend/config/models.py:129,183,298`; `backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86, gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py, stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter: gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`; `DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`; `DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`; `PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`, `PORTABILITY.md`, `web/README.md`.

Changelog
- 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs (FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor re-verified against source before writing (catching the native-WASB partial-wiring + `ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local), multi-provider GPU, and first-class Observability directives.

8. user (2026-06-21T13:52:20.162Z)

Cross-Platform Desktop App ? "Plug in your card, get your highlights"

> **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) · **Owner:** avidullu · **Created:** 2026-06-18 · **Last updated:** 2026-06-18
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner greenlights P0).
> **Relation to existing docs:** **peer-of** [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop` ComputeBackend** made into a product) · **realizes the L0 tier** of [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) · **consumes** [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-clean model is one answer to the compute crux) · bound by [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) licensing guardrails.
> **Honesty note:** every claim is tagged `[verified]` (checked against code/measured), `[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ? this doc scopes; it does not ship. Not legal advice.

---

0. TL;DR
Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg stitch, already runnable fully-offline via [`tools/generate_highlights.py --skip-ai-handoff`](../tools/generate_highlights.py)) as a **cross-platform desktop app** for **non-technical recreational players**: plug in the camera's USB/SD card, click once, get back **only the highlight reels** ? fully on-device, **no upload**, **no copy of the GBs of 4K originals**. This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing leaves the machine") tier of `VIDEO_LOCALITY_MODEL.md`.

The make-or-break unknown is compute on a typical enthusiast laptop ? which often has **no discrete GPU**. WASB runs at **~3.4× real-time on a laptop RTX 2070 Super** `[verified, measured]`; on CPU it is far slower, and Apple Silicon has **no CUDA** at all. **P0 is a feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app is built.** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster owned model (ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or a tiered "fast-if-GPU, slow-but-works-if-CPU" design. **Everything past P0 is gated on that result.**

1. Problem & goal

Why now / the user. Target users are **recreational sports enthusiasts, not tech-savvy**. Their match videos are **huge** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (`VIDEO_LOCALITY_MODEL.md` §0: 10 GB ? ~1 h at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is **kilobytes of timestamps ? a highlight reel** (`VIDEO_LOCALITY_MODEL.md` §1: *"timestamps are the product; the video is dead weight"*).

The concrete outcome ("good" looks like this):
1. User plugs the GoPro/phone SD card or camera USB into their laptop.
2. The app detects the card, lists the match videos on it.
3. User clicks **Generate Highlights**.
4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? **100% locally**.
5. The reel(s) land in a user-chosen folder. **The originals are never copied off the card and never bloat any drive.**

No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (`COMMERCIALIZATION` C4).

Read to get current: `VIDEO_LOCALITY_MODEL.md` (the L0/locality framing + OQ5 packaging seed), `PLATFORM_ARCHITECTURE.md` §3.2 (the ComputeBackend seam), [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (the detector ABC that already anticipates a Linux-native runner), `tools/generate_highlights.py` (the local entrypoint this app wraps).

2. Decisions locked

#	Decision	Source / date	Implication
D1	**Default path is 100% on-device, no upload, no Gemini.** The reel is produced from the local pipeline (`--skip-ai-handoff`).	This doc · 2026-06-18; matches `VIDEO_LOCALITY_MODEL` L0	The app must never *require* a network call. Gemini is an opt-in toggle only.
D2	**Never copy the originals.** The app reads source video in place (off the card / a chosen folder) and writes **only** reels + a tiny report to a user folder.	This doc; mirrors `generate_highlights.py` non-destructive `_atomic_publish` discipline	No "import library" that duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place.
D3	**Drop the WSL2 dependency; run the detector natively per-OS.**	This doc; enabled by the `DetectorRunner` ABC `[verified]`	Windows-native (no WSL), Linux+NVIDIA (CUDA directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is **not** the shipped path.
D4	**Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264).	`COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO_LOCALITY_MODEL` OQ5 (*"ffmpeg LGPL build = licensing-clean"*)	WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`). The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See §3.4.
D5	**Reuse the existing pipeline verbatim; add only the shell + a native runner + device-IO.**	`backend/pipeline/detectors/base.py` reuse contract `[verified]`	No re-architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging.
D6	**Feasibility is gated on P0.** No app shell, no packaging work starts until the compute spike returns a verdict.	This doc; `decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS DECIDE)	P0 is the only un-gated deliverable; P1?P3 are owner-greenlit *after* P0 numbers.

3. Foundation ? research / prior art `[design]` / `[researched]`

3.1 The compute reality (the crux) ? what we know vs must measure

- **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's **RTX 2070 Super Max-Q (8 GB)**, WASB inference is the pipeline driver at **~3.2?3.4× real-time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC) ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
- **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4× RT is ~34 GPU-min *with* a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and could be 10×+ worse ? possibly **hours per clip**, which would kill the one-click promise.
- **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal) backend *may* run them, but op-coverage and speed on the WASB model are **unverified** `[design]`. CPU fallback on Apple is also unverified.
- **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-device answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch / GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to **train with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible answer if the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a referendum on whether the desktop product needs the owned-model track.**

3.2 The de-WSL native runner is already designed-for `[verified]`

The detector layer was deliberately built to make this port low-regret. [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents, verbatim, an **"Optional Linux-native path (the main de-risk goal)"**: a `LinuxNativeRunner`/`ONNXRunner` that *"Skip[s] all wsl / _stage / conda activate, load[s] weights locally (torch or onnx)? write[s] the exact same CSV format so `trajectory_to_action_windows` works verbatim."* The WSL plumbing lives in a separate `WslCommandMixin`, explicitly so a native runner *"must not inherit WSL plumbing."* The actual inference math is already isolated in [`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules, emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing brain (`trajectory_to_action_windows`) and stitch are untouched.

3.3 Compute backends to evaluate in P0

Path	OS	Toolkit	Status
CUDA-native	Windows-native, Linux+NVIDIA	torch+CUDA (no WSL)	proven in WSL `[verified]`; native = port
Apple MPS	macOS (Apple Silicon)	torch MPS / Core ML export	**unverified ? P0 must measure**
CPU	all (the no-GPU laptop)	torch-CPU / **onnxruntime** (often fastest on CPU)	**unverified ? P0 must measure; the gating number**
Smaller/exported model	all	ONNX/Core ML/ExecuTorch, quantized	deferred to owned-model track; P0 informs whether it's *needed*

3.4 App-shell prior art & licensing landscape `[researched]`

- **App shell options:** **Tauri** (Rust + system webview; ~3?10 MB binaries; MIT/Apache) vs **Electron** (bundled Chromium; ~100?150 MB; MIT) vs **CLI + installer** (wrap `generate_highlights.py`; smallest effort). All shell licenses are clean.
- **Backend packaging:** the Python/FastAPI backend bundles via **PyInstaller** or a frozen venv as a local "sidecar." Packaging was already modernized ? **PEP 621 `pyproject.toml` + an `indexer` console entry point** shipped (`DEFERRED_HARDENING_PLAN` #7, DONE, PR #167) `[verified]` ? so a CLI MVP has a real head start.
- **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path uses **libx264 (GPL)**. Redistributing that inside an installer would impose GPL on the bundle. **Mitigation (D4):** ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer hardware H.264 anyway (faster, no CPU stitch tax).
- **Weights:** WASB-SBDT = MIT `[verified]` (`tracknet-wasb-wsl2-decision`), redistributable.

4. Design / architecture `[design]`

4.1 The one-click flow (and what code each step reuses)

```
```

```
[card inserted]
    ?  (NEW) per-OS device-insert watcher ? mount path
    ?
[list videos on the card]   ? glob video exts; reuse _VIDEO_EXTS from generate_highlights.py
    ?
[make on-device proxy]      ? ffmpeg downscale (hardware encoder); reuse the proxy + --detect-from split (#205)
    ?
[detect: WASB native]       ? (NEW) NativeWasbRunner (DetectorRunner subclass) ? NO WSL; same CSV contract
    ?
[windowing ? rallies]       ? REUSE trajectory_to_action_windows + improved milestone windowing (R1: cap=6 + R4)
    ?
[stitch reel from ORIGINAL] ? REUSE VideoCompiler (backend/pipeline/stitcher/compiler.py); read original in place
    ?
[export reel + tiny report to a user folder]   ? reuse _atomic_publish non-destructive write; NEVER copy originals
```

The detector runs on a **proxy**; the stitch reads the **original on the card** (the `--detect-from` split, already shipped `[verified]` in `generate_highlights.py`). Rally timestamps transfer 1:1 because the proxy shares the timeline (the collector frame-identical proxy contract, `VIDEO_LOCALITY_MODEL` §1).

4.2 Reuse map ? what's NEW vs REUSED

Concern	Status	Code
Trajectory?rally windowing	**REUSE** `[verified]`	`tracknet_runner.trajectory_to_action_windows` (model-agnostic staticmethod)
Stitch / reel compile	**REUSE** `[verified]`	`backend/pipeline/stitcher/compiler.py::VideoCompiler` (pure ffmpeg)
Local orchestration (proxy?detect?stitch, non-destructive, resumable, `--skip-ai-handoff`, `--detect-from`)	**REUSE** `[verified]`	`tools/generate_highlights.py`
Detector runner contract	**REUSE** `[verified]`	`backend/pipeline/detectors/base.py::DetectorRunner` ABC
WASB inference math	**REUSE (lift out of WSL)** `[verified]`	`backend/pipeline/detectors/wasb_infer.py`
Native (de-WSL) WASB runner, per-OS	**NEW** `[design]`	`NativeWasbRunner` ? sibling of `wasb_runner.py`, no `WslCommandMixin`
USB/SD insert detection + video listing	**NEW** `[design]`	per-OS: Win WMI/`WM_DEVICECHANGE`, macOS DiskArbitration, Linux udev/udisks2
App shell / one-click UI	**NEW** `[design]`	Tauri vs Electron vs CLI (§3.4; P2 decides)
Packaging + code-signing per OS	**NEW** `[design]`	PyInstaller/MSIX (Win), .dmg+notarize (mac), AppImage/Flatpak (Linux) ? P3

4.3 Where this sits in the platform architecture

This **is** the `local` / `LocalDesktop` **ComputeBackend** of `PLATFORM_ARCHITECTURE.md` §3.2, realized as a shippable product instead of a server worker. The control-plane/data-plane split degenerates cleanly to "all on one machine." Nothing here forecloses the hosted SaaS path ? it's the privacy-max end of the same spectrum, and a genuine differentiator (no competitor offers true on-device compute except SwingVision, which proves it's commercially viable ? `PLATFORM_ARCHITECTURE` §intro).

4.4 App-shell recommendation (to confirm at P2)

Lean: **start with a CLI + installer MVP** (wraps the existing `indexer`/`generate_highlights.py` entry point ? fastest way to validate packaging, signing, USB-detect, and the native runner end-to-end on all three OSes), then **graduate to a Tauri GUI** for the non-technical user (tiny download fits the "huge-video laptops, slim app" frame; the Python backend runs as a sidecar the Tauri front-end calls over localhost). Electron is the fallback if webview inconsistencies bite. This mirrors the repo's "simplest thing first, graduate on evidence" bias.

5. Risk table `[design]`
Risk	Severity	Mitigation / where decided
CPU/MPS too slow on a no-GPU laptop (kills the one-click promise)	**make-or-break**	**P0 spike measures it before anything is built** (§6, §7-P0). Fallbacks: tiered fast/slow UX, or the owned smaller model.
ffmpeg GPL contamination via libx264 in the installer	high (legal)	D4: LGPL build + openh264/hardware encoders only.
Code-signing friction (mac notarization mandatory; Win SmartScreen without EV cert)	medium	P3 scopes certs/notarization per OS; budget the Apple Developer Program + a Windows cert.
Privacy regressions (the on-device promise is the *selling point*)	medium	D1/D2: no upload, no original-copy, no required network. Make the "nothing leaves your machine" guarantee testable (assert no outbound calls in default mode).
Per-OS device-insert flakiness / removable-media edge cases (card pulled mid-run)	medium	P1/P2 handle gracefully; never write to the card; fail safe.
Support burden for non-technical users (drivers, "it didn't detect my card")	medium	favors a polished GUI + clear errors (reuse the actionable-error discipline already in the WSL adapter).

6. Honest limits ? what this does NOT do
- **A DRAFT/DONE status here proves nothing about runtime feasibility.** The doc is a scope; only the P0 numbers decide whether the product is buildable on commodity hardware.
- **If CPU-only is too slow on a typical laptop, the simple "wrap the pipeline" path does not exist** ? it becomes "ship a smaller owned model" (a much larger effort, ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or "GPU-tier only" (shrinks the addressable market). The doc names this honestly rather than assuming the wrap is free.
- **No accuracy claim changes.** The desktop app inherits exactly the current rally-detection quality (R1 milestone: precision is still the open gap per `RALLY_DETECTION_GAP_CLOSURE.md`); packaging does not improve detection.
- **Gemini-quality reels need the optional online toggle.** The fully-local default uses `--skip-ai-handoff`; users wanting the Gemini brain accept that ?1280px chunks leave the machine (consent language, `VIDEO_LOCALITY_MODEL` OQ3).
- **macOS support is contingent on P0's MPS/CPU verdict** ? it is not assumed.

7. Deliverables & progress tracker   ? **source of truth**
Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands. **Everything below is owner-gated; only this doc (the scoping PR) is in flight today.**

ID	Deliverable	Depends on	Gated?	Status	PR
D0	**This scoping doc** (`DESKTOP_APP_PLAN.md` + README index)	?	No	?	*(this PR)*
P0	**Compute-feasibility spike (make-or-break).** Measure WASB throughput for a ~10-min clip on: (a) CPU-only (torch-CPU & onnxruntime), (b) Apple MPS, (c) confirm CUDA-native (no WSL). Output: a wall-clock table + a go/no-go verdict + "do we need a smaller model?" recommendation. **No app code.**	D0	**Owner greenlight**	?	?
P1	**De-WSL native detector port** (`NativeWasbRunner`, per-OS): CUDA-native (Win/Linux), MPS/CPU fallback, identical `Frame,Visibility,X,Y` CSV; reuse windowing verbatim; healthcheck per backend.	P0 verdict	Yes	?	?
P2	**Desktop app shell + one-click flow**: USB/SD detect ? list ? proxy ? native detect ? stitch ? export; shell choice (CLI MVP ? Tauri) confirmed; "no upload / no original-copy" enforced & testable.	P1	Yes	?	?
P3	**Packaging + code-signing installers** per OS (Win MSIX/Authenticode, mac .dmg + Developer-ID notarization, Linux AppImage/Flatpak); LGPL ffmpeg bundling; MIT/Apache/BSD audit of the bundle.	P2	Yes	?	?

8. Open questions ? owner / external
Blocking gates (owner decides before the dependent phase):
1. **(gates P1?P3) Greenlight P0?** ? the spike is cheap and decides everything. Recommend: yes, run P0 next.
2. **(gates P1) If CPU is too slow, which fork?** tiered GPU-fast/CPU-slow UX · invest in a smaller exported owned model (`OWNED_MODEL_IMPLEMENTATION_PLAN`) · GPU-tier-only product (narrower market).

3. **(gates P2) App shell:** confirm CLI-MVP-then-Tauri, or go straight to a GUI / Electron.
4. **(gates P3) Distribution & signing budget:** Apple Developer Program (mandatory for notarization), a Windows code-signing cert (EV for clean SmartScreen) ? owner cost decision.

Nice-to-knows:
5. OS coverage priority for v1 (Windows-first? given the owner's box and the academy beachhead).
6. Auto-update strategy (Tauri updater / Sparkle / none for the pilot).
7. Does the desktop app reuse the optional Gemini toggle at all, or stay strictly local for v1?

9. Definition of done
This subproject is DONE (? flip Status, archive) when:
- [ ] **P0** produced a measured throughput table (CPU / MPS / CUDA-native) and an owner-accepted go/no-go verdict (incl. whether a smaller model is required). *(P0 alone is a valid stopping point if the verdict is "not feasible without the owned model" ? then this doc hands off to `OWNED_MODEL_IMPLEMENTATION_PLAN`.)*
- [ ] **P1** the native (de-WSL) detector runs on ?1 GPU OS and ?1 CPU/MPS path, producing byte-compatible trajectories (regression vs the WSL runner on a shared clip).
- [ ] **P2** a non-technical user can insert a card and get a reel with no terminal, no upload, and no copied original ? verified end-to-end on ?1 OS.
- [ ] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes an MIT/Apache/BSD (+ LGPL-ffmpeg) license audit.
- [ ] §7 rows all ? and the headline feasibility verdict is recorded for posterity.

10. References
Internal code anchors `[verified]`: [`tools/generate_highlights.py`](../tools/generate_highlights.py) (local orchestration; `--skip-ai-handoff`, `--wasb-stream`, `--detect-from`, `_WSL_DETECTORS`, `_atomic_publish`) · [`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) (`DetectorRunner` ABC + the documented Linux-native de-risk path + `WslCommandMixin`) · [`backend/pipeline/detectors/wasb_runner.py`](../backend/pipeline/detectors/wasb_runner.py) (the WSL adapter to replace) · [`backend/pipeline/detectors/wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (the torch inference core to lift native) · [`backend/pipeline/detectors/tracknet_runner.py`](../backend/pipeline/detectors/tracknet_runner.py) (shared `trajectory_to_action_windows`) · [`backend/pipeline/segmenters/wasb_hybrid.py`](../backend/pipeline/segmenters/wasb_hybrid.py) · `backend/pipeline/stitcher/compiler.py::VideoCompiler` · `pyproject.toml` + `indexer` console entry (#167).
Internal docs: [`VIDEO_LOCALITY_MODEL.md`](VIDEO_LOCALITY_MODEL.md) (L0 fully-local tier; OQ5 packaging) · [`PLATFORM_ARCHITECTURE.md`](PLATFORM_ARCHITECTURE.md) §3.2 (ComputeBackend seam) · [`OWNED_MODEL_IMPLEMENTATION_PLAN.md`](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (export-clean on-device model) · [`COMMERCIALIZATION.md`](COMMERCIALIZATION.md) (C4 paid-LLM no-train; MIT/Apache/BSD-only) · [`DEFERRED_HARDENING_PLAN.md`](DEFERRED_HARDENING_PLAN.md) (#7 packaging, DONE) · [`TRACKNET_WSL_SETUP.md`](TRACKNET_WSL_SETUP.md) (the WSL setup being superseded for shipped builds).
Measured source: `current-state.md` 2026-06-18 `COST_SCALING.md` checkpoint (WASB ~3.4× RT on RTX 2070 Super Max-Q, peak VRAM ~5.7?6.0 GB).
External (to validate at P0/P2/P3): PyTorch MPS backend · onnxruntime (CPU/CoreML/DirectML execution providers) · Tauri vs Electron sizing · ffmpeg LGPL build / openh264 · Apple Developer-ID notarization · Windows Authenticode/MSIX.

Changelog
- `2026-06-18` ? Created. Scoping doc for the cross-platform desktop app + the de-WSL native-compute port; all `[design]`. §7 P0 (compute-feasibility spike) named the make-or-break gate. Owner-gated; nothing built.

9. assistant (2026-06-21T13:52:26.679Z)

I have excellent grounding now. D10 is clear and the appliance architecture is well-documented.

Let me check a couple more engine facts and the friendly/multilingual doc, then do web research.

10. user (2026-06-21T13:52:28.106Z)

```
        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig
        _enforce_startup_or_exit(global_config)  # M5: fail fast on an incoherent ACTIVE profile

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
        db_instance = Database(db_path)

        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
        # the executor is in-process. Mark them failed so pollers see a terminal state.
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to scratch.
            # The local-existence fast-fail only applies to a local input (stat the RESOLVED key, not the
            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a crash.
            storage_cfg = getattr(global_config, "storage", None)
            try:
                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
            except ValueError as e:
                print(f"[FAIL] {e}")
                return
            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
```

11. user (2026-06-21T13:52:29.226Z)

```
.claude\worktrees\bhi-execpolicy\docs\I18N_PLAN.md
.claude\worktrees\bhi-execpolicy\docs\UI_PROPOSAL.md
.claude\worktrees\agent-a771ff1d85e18f2ed\docs\I18N_PLAN.md
.claude\worktrees\agent-a771ff1d85e18f2ed\docs\UI_PROPOSAL.md
.claude\worktrees\tier1-windowing\docs\I18N_PLAN.md
.claude\worktrees\tier1-windowing\docs\UI_PROPOSAL.md
.claude\worktrees\w2-safeguard\docs\I18N_PLAN.md
.claude\worktrees\w2-safeguard\docs\UI_PROPOSAL.md
.claude\worktrees\hardchop\docs\I18N_PLAN.md
.claude\worktrees\hardchop\docs\UI_PROPOSAL.md
.claude\worktrees\boxhill17\docs\I18N_PLAN.md
.claude\worktrees\boxhill17\docs\UI_PROPOSAL.md
```

```
.claude\worktrees\pr-stitch\docs\I18N_PLAN.md
.claude\worktrees\pr-stitch\docs\UI_PROPOSAL.md
.claude\worktrees\gap-doc\docs\I18N_PLAN.md
.claude\worktrees\gap-doc\docs\UI_PROPOSAL.md
.claude\worktrees\resweep\docs\I18N_PLAN.md
.claude\worktrees\resweep\docs\UI_PROPOSAL.md
.claude\worktrees\recdoc\docs\I18N_PLAN.md
.claude\worktrees\recdoc\docs\UI_PROPOSAL.md
docs\I18N_PLAN.md
docs\UI_PROPOSAL.md
```

12. user (2026-06-21T13:52:30.410Z)

```
556:def main():
562:    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API server")
```

13. assistant (2026-06-21T13:52:33.416Z)

Now let me look at how the server is launched (whether it opens a browser today) and read the FRIENDLY doc, then start web research.

14. user (2026-06-21T13:52:37.665Z)

```
            print(f"[FAIL] Video file not found: {args.video_path}")
            return
        local_video = storage.acquire_local_input(args.video_path, storage_cfg)

        video_id = compute_video_id(local_video)
        was_reingest = db_instance.get_video(video_id) is not None

        # Validate (with-context variant surfaces ffprobe_meta for the report)
        print("[CLI] Validating...")
        val_results, val_context = registry.run_all_with_context(local_video, global_config)
        failures = [r for r in val_results if not r.passed]

        # Save video status
        status = "failed" if failures else "processed"
        db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in val_results])

        seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None), "default_segmenter", "motion")
        segmenter_actual = None
        segmentation_ran = False
        try:
            print("\n" + "="*40)
            print("VALIDATION SUMMARY")
            print("="*40)
            for r in val_results:
                status_symbol = "[PASS]" if r.passed else "[FAIL]"
                print(f"{status_symbol} {r.validator_name}: {r.message}")
            print("="*40 + "\n")

            if failures:
                print("[FAIL] Video failed checks. Indexing halted.")
                return

            # Index
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return
```

```
            print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
            segmenter_actual = seg_name
            play_wm, cand_wm = db_instance.segment_watermarks(video_id)
            segmenter = segmenter_cls(db_instance, global_config)
            result = segmenter.process_video(video_id, local_video, max_frames=args.max_frames)
            segmentation_ran = True

            if isinstance(result, Failure):
                print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
                print("[FAIL] Indexing halted for video.")
                segments = []
            else:
                segments = result.value if hasattr(result, "value") else result
                print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside database: {db_path}")
            # Destroy-AFTER-confirm: keep new on success, restore prior on empty/failure.
            _finalize_reingest(video_id, segments, play_wm, cand_wm)
        finally:
            # Always emit the per-video observability report. Never raises.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=args.video_path, config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
```

15. user (2026-06-21T13:52:38.195Z)

Friendly & Multilingual ? non-tech-savvy UX + Hindi/Kannada localization

> **Status:** `DRAFT` ? owner-gated (implementation starts only on explicit approval) · **Owner:** Avi Dullu · **Created:** 2026-06-21 · **Last updated:** 2026-06-21
> **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE).
> **Tracking anchors:** §7 progress tracker is the source of truth; indexed in `docs/README.md`; thread in `NEXT_STEPS.md`; pointer in engine `docs/DOC_STATUS.md`; memory `current-state` + `next-session-ux-localization`.
> **Relation to existing docs:** this is the **implementation project** for two engine design docs ? it *adapts & supersedes-in-practice* `badminton-highlight-indexer/docs/I18N_PLAN.md` (localization) and the UX half of `?/docs/UI_PROPOSAL.md` (non-tech-savvy console). Those stay the canonical *design*; this doc carries the *tracked execution* against today's surfaces.
> **Honesty note:** each claim tagged `[verified]` (checked against code this session), `[researched]` (needs sign-off), or `[design]` (proposed). Not legal advice ? the OFL font question (D-FONT) is owner/counsel-gated.

---

0. TL;DR
A coach or parent in Bangalore should land on KhelSutra, feel at home, set up their camera with confidence, and get highlights ? **in Kannada or Hindi**, with **zero hand-holding**. This project isolates that goal into one tracked workstream-pair: **(A)** make the UI super friendly for non-tech-savvy users, and **(B)** localize to Hindi + Kannada. Both surfaces (`web/` React SPA + `site/` vanilla marketing) have **zero i18n today** `[verified]`, so the foundation ? shared `locales/{en,kn,hi}` catalogs + a negotiation contract + CI parity ? is the real deliverable; translations follow. **The single most important caveat:** translations are **human-reviewed** (Gemini drafts are *inference only, never training* ? the standing guardrail), and Kannada/Hindi rendering is **blocked on the OFL font decision (D-FONT)**. The cheapest unblocked high-impact first slice is the **SPA i18n scaffold + language switcher (en-only)** ? it locks the shared catalog/negotiation contract and adds the render-test safety net the SPA currently lacks.

1. Problem & goal

15 **Why now.** The public site (khelsutra.guru) is live, clean, honest, and tested. The PMF beachhead is **Bangalore badminton academies**; many coaches, parents, and players are far more comfortable in **Kannada** than English, and most are **not highly tech-savvy** `[verified ? design intent]` (`UI_PROPOSAL.md:25,28`, `I18N_PLAN.md:10-13`, `PMF_MARKET_RESEARCH.md:116`, `KHELSUTRA_PRODUCT_OVERVIEW.md:13/50/61`). An English-only, knob-heavy product adds friction at the exact point of first contact.
16
17 **Concrete outcome ("good" looks like):** a Kannada/Hindi-comfortable, phone-first, non-technical coach/parent can ? without help ? (1) understand what the product does and how to record reliably, (2) request access / use the console, (3) pick rallies and get a reel, and (4) share it on WhatsApp, **entirely in their language**.
18
19 **Read to get current:** this doc's §3?§4; the engine design `I18N_PLAN.md` (canonical i18n design) + `UI_PROPOSAL.md` §2 (personas, the 6 alpha screens, "teach recording *before* upload"); `VIDEO_RECORDING_GUIDELINES.md`; this repo's `PER_RALLY_SELECTION.md` (the honest SPA scope) + `REQUEST_ACCESS_FORM.md`.
20
21 ## 2. Decisions locked (structural ? owner directive 2026-06-21)
22 | # | Decision | Source / date | Implication |
23 |---|----------|---------------|-------------|
24 | D1 | **Isolate as ONE tracked project** spanning both workstreams (A: UX, B: kn/hi l10n) | owner, 2026-06-21 | This doc; §7 = source of truth; archived on DoD |
25 | D2 | **Home = `khelsutra/docs/`** (the implementation repo) | owner-gated tracked-doc convention | `web/` + `site/` both live here; §7 tracks khelsutra PRs; engine P2 backend-l10n is a cross-repo deferred row |
26 | D3 | **Adapt `I18N_PLAN.md` to today's surfaces:** P1 (vanilla `t()` shim) ? `site/`; P3 (react-i18next) ? `web/`; **drop the dead `backend/api/static` retrofit** | `[verified]` the doc targets `backend/api/static` (`I18N_PLAN.md:25,30,156,191`) which exists only in the engine repo, not here | The doc's framework-agnostic *foundation* (catalog format + negotiation + ICU plurals + CI) survives unchanged; only the wiring re-points |
27 | D4 | **Engine P2 backend-localization stays owner-gated + cross-repo** (not in this DoD) | `CLAUDE.md` guardrails | Dynamic engine error/report text localizes later; tracked as a deferred row |
28 | D5 | **Archive trigger = en+kn+hi live on BOTH surfaces** (+ the non-tech UX baseline), then run the engine `DOC_STATUS.md §2` archival checklist | owner, 2026-06-21 | See §9 |
29 | D6 | **D-LOCALES: `en` + `kn` for v1, `hi` next; always-fallback to `en`** | owner, 2026-06-21 | Kannada-first beachhead; `hi` = catalog-only (both 2-form CLDR) |
30 | D7 | **D-FONT: OFL-for-media carve-out ? self-host Noto Sans Kannada/Devanagari** (counsel may still confirm) | owner, 2026-06-21 | Unblocks B4/B5/A3 + the kn/hi reel watermark; an explicit exception to the MIT/Apache/BSD code guardrail (fonts = media) |
31 | D8 | **D-SURFACE-FIRST: lead with the `web/` SPA scaffold; SKIP the dead static-UI retrofit** | owner, 2026-06-21 (via B1 go-ahead) | B1 is greenlit; site recording-education localized early |
32 | D9 | **D-CATALOG-HOME: co-locate `locales/` + CI in `khelsutra`** (shared, consumed by SPA + site) | owner, 2026-06-21 (via B1 go-ahead) | No cross-repo coupling |
33
34 *Only **D-BACKEND-DEPTH** (§8.3) remains open ? and it's a deferred, cross-repo engine concern (B7), not a v1 blocker.*
35
36 ## 3. Foundation ? research / prior art
37 **Current-state facts `[verified]` this session** (adversarially checked against the working tree):
38 - **`web/` SPA: zero i18n.** `web/package.json` deps are only `react`/`react-dom`; no `i18next`/`react-intl`/`formatjs`; no `locales/` dir; no `t()`/`useTranslation`/`data-i18n` anywhere. User-facing English is hardcoded in `.tsx` (e.g. `QueryBar.tsx:46-47,56,72,76,78`; `App.tsx:115,211,224`) ? ~50 literals + ~16 derived labels.
39 - **`site/` marketing: vanilla, zero i18n.** Three `<html lang="en">` files (`public/{index,capture,own-model}.html`) with inline CSS, no framework; the only scripts are a CF analytics beacon (`index.html:595`) + one inline form/reel IIFE (`index.html:597-633`). `server.js` hardcodes English responses; no locale handling.
40 - **Recording-education is *not* greenfield.** The homepage "Record it right" gist (`index.html:324-364`) + the `/capture` checklist (`capture.html`, routed `server.js:51`, guarded `assets.test.js:94`) already ship the user-facing best-practices surface. *Unshipped:* the empirical-threshold framing and the programmatic pre-flight validator (a code change, not a

page).
- **Persona is design intent, not built.** kn/hi + zero-knobs are **proposed/owner-gated**; the app is English-only today (`KHELSUTRA_PRODUCT_OVERVIEW.md:61`). It's Kannada-**first**, Hindi-**next** ? not Kannada-only.

Localization prior art `[researched]`:
- **CLDR plural categories:** Hindi = `{one, other}`; Kannada = `{one, other}`. Both are 2-form (same shape as English) ? once ICU MessageFormat is in the catalog, adding `hi`/`kn` plurals is **catalog-only**, no code. (ICU still required because English `count>1?'s':''` hand-rolling ? present today in the SPA ? doesn't generalize.)
- **react-i18next** (web/): `i18next` + `react-i18next` + `i18next-browser-languagedetector` + an ICU/plural post-processor; lazy-load per-locale JSON; persist via `localStorage`.
- **Shared catalog + tiny shim** (site/): the SAME i18next-flavored JSON consumed by a ~30-line `t(key, vars)` shim that walks `data-i18n` attributes ? no build step, no framework, mirrors i18next interpolation/plural rules for the 2-form languages.
- **Noto Sans Kannada / Noto Sans Devanagari** are **OFL-1.1**; OFL permits commercial bundling/redistribution (fonts = media, not code/weights). Self-host (e.g. `@fontsource`) with `font-display: swap` + subsetting. **Gated on D-FONT.**
- **Non-tech-savvy / HCI4D patterns:** icon **+ text** (never icon-only), non-color selection indicators, large touch targets, mobile-first, WhatsApp as the share channel, a prominent persistent language switcher.

4. Design / architecture
Two surfaces, one shared source of truth.

```
locales/                      # shared catalogs ? co-located in khelsutra (D-CATALOG-HOME, see §8)
  en/  common.json            # source of truth ? devs author keys here
  kn/  common.json            # Kannada  (Gemini-draft ? human-review)
  hi/  common.json            # Hindi    (next ? the extensibility proof)

web/   ? react-i18next        # I18N_PLAN P3: provider in main.tsx; useTranslation in components
site/  ? t(key,vars) shim      # I18N_PLAN P1 retargeted: data-i18n attrs over the static HTML
```

- **Negotiation contract (one order, both surfaces):** `persisted pref (localStorage) ? switcher / ?lang=<code> ? navigator/Accept-Language ? "en"`. Supported locales in a single module; anything else falls back to `en`; **a missing key degrades to English, never to a raw key.**
- **Keys, never prose** ? `t("compile.button")`, with ICU for interpolation/plurals: `"{count, plural, one {# rally} other {# rallies}}"`. This replaces the SPA's current hand-rolled `Segment${count>1?'s':''}`.
- **Translation pipeline (guardrail-clean):** dev authors `en` keys ? `scripts/i18n_draft.py`-style Gemini **draft** (inference only, never training; flags every value for review) ? **owner/native-speaker review** ? CI key-parity ? merge. Gemini never touches `en`, never invents keys.
- **CI enforcement (keeps it "by design"):** (1) **key-parity** ? every locale has exactly the `en` key set (drop a stub `ta.json` ? CI lists every gap = the extensibility proof); (2) **no-bare-strings** ? flags user-facing literals not routed through `t()` (curated allowlist to start).
- **Fonts:** bundle Noto Sans Kannada + Devanagari **iff D-FONT = OFL carve-out**; this also unblocks a Kannada/Hindi **reel watermark** (engine `WatermarkConfig.font_path` already takes a path ? config, not code).

Non-tech-savvy UX levers (ranked impact × effort, grounded in personas; surface-tagged):

Lever	Surface	Impact	Effort	Note
Plain-language copy ? strip engine jargon, keep honest caveats	both	high	small	banners + footer carry backend vocabulary today

74 | Localized **WhatsApp share** for the downloaded reel | both | high | small | WhatsApp is *the* channel for this audience |
75 | **Language switcher** persisted via a shared localStorage key | both | high | small | site promises kn/hi but ships no switcher |
76 | Localize **recording-education** (capture + gist) kn?hi | site | high | medium | gates input quality; the highest-leverage kn content |
77 | Mobile-first touch targets; tooltip-text ? visible | web | high | medium | tiny targets + desktop layout + invisible tooltips on phones |
78 | Icon **+ text** + non-color selection indicator | web | medium | small | never icon-only |
79 | Friendly engine-error mapping over raw FastAPI `detail` | web | medium | medium | `CompileResult` passes engine text untranslated |
80 | Plain-language framing for the Request-Access GPU/storage Qs | site | medium | small | "GPU"/"S3/GCS" are intimidating |
81 | Background-processing reassurance + status as icon chips | web | medium | small | honest expectations on slow uplinks |
82 | Fix English-source recording gaps (a diagram) before translating | site | medium | medium | the diagram is reusable across all 3 languages |
83 | Tap-only filter controls alongside the English query bar | web | medium | large | the `parseQuery()` parser is English-grammar only |
84 | Keep the honest *today-vs-roadmap* split through translation | both | medium | small | never soften "experimental"/"not available yet" |
85
86 **Reuse map (what's new vs reused):** *Reused* ? the i18next-flavored JSON + negotiation contract + ICU plurals + key-parity/no-bare-strings CI + draft?review pipeline (all from `I18N_PLAN.md`); the shipped recording pages; the SPA's `lib/format.ts`/`lib/query.ts`; `site/src/assets.test.js` + the SPA's vitest harness. *New* ? `locales/` catalogs; the SPA react-i18next provider + switcher; the site `t()` shim + `data-i18n` wiring; Noto font bundle (gated); a jsdom render-test env for the SPA.
87
88 ## 5. Privacy / scope note
89 i18n touches **presentation only** ? no new personal-data collection, no biometrics, minors-excluded posture unchanged. WhatsApp share opens the user's own client with a pre-filled message; it sends nothing server-side. Gemini-draft translation is inference, not training (no user data in the loop).
90
91 ## 6. Honest limits ? what this does NOT do
92 - A green CI / DONE status proves **key-parity + render + no-bare-strings**, **not** translation *quality* ? that's human-gated and owner-paced.
93 - Kannada/Hindi will **not render** until the Noto fonts land (D-FONT); until then kn/hi catalogs exist but display tofu/fallback.
94 - The SPA is **job-tethered**; localizing **dynamic engine errors/report text** needs engine P2 (owner-gated, cross-repo, D-BACKEND-DEPTH) ? out of this DoD.
95 - The query bar stays **English-grammar parsing**; kn/hi users rely on tap-only controls (a later, larger slice), not free-text Kannada queries.
96 - **Mobile-*perfect*** layout is an alpha non-goal (usable-on-phone is in scope; pixel-perfect is not).
97
98 ## 6.5 Testing strategy ? automatic language negotiation (rigorous, owner-required)
99 **The bar (owner, #41):** a user whose **computer/browser language is Kannada or Hindi must get the respective version automatically** ? that is the core promise and it is **proven by tests at every layer, never assumed.** Auto-selection IS the negotiation order (persisted pref ? `?lang=` ? browser/`Accept-Language` ? `en`); each step gets a test.
100
101 **Negotiation matrix ? every row is an automated test:**
102
103 | # | Signal under test | Setup | Expected | Proven in |
104 |---|---|---|---|---|
105 | N1 | **Browser/OS = Kannada** | no persisted pref, no `?lang`, `navigator.languages=["kn-IN", ?]` | resolves to **kn** ? Kannada content (once the kn catalog ships) | **B1** detection · **B5** content |
106 | N2 | **Browser/OS = Hindi** | `navigator.languages=["hi-IN", ?]` | resolves to **hi** ? Hindi content | **B6** (hi added to the supported set + catalog) |
107 | N3 | **`?lang=` override** | `?lang=kn`, browser = en | kn | B1 |
108 | N4 | **Persisted pref wins** | `localStorage=kn`, browser = en, no `?lang` | kn (a

returning user keeps their choice) | B1 |
N5	**Region subtag normalizes**	`navigator.language="kn-IN"`	base **kn** (not a "kn-IN" miss)	B1
N6	**Unsupported language**	`navigator="ta-IN"` (Tamil, not yet supported)	**en** fallback ? never a locale we can't serve	B1
N7	**Missing key in active locale**	kn active, a key absent from `kn/common.json`	**en** fallback for that key (no raw key shown)	B1/B5
N8	**Static site (vanilla shim)**	`Accept-Language: kn` / `?lang=kn` on a `site/` page	kn-rendered page (same order as the SPA)	B3
N9	**Backend (engine P2)**	`Accept-Language: kn` to the API	kn customer/report text	B7 (owner-gated)

Levels & tooling:
- **SPA (`web/`):** jsdom tests stub `navigator.language`/`navigator.languages`, `localStorage`, and the `?lang` query ? assert `i18n.language`/`resolvedLanguage` + rendered text per the matrix. *(B1 lands N1/N3?N7 at the **detection** level now ? `kn` is supported; the kn/hi **content** assertions ride the catalogs in B5/B6.)*
- **Static site (B3):** the shared `t()` shim's negotiation is unit-tested the same way; plus a server test that `Accept-Language: kn`/`?lang=kn` returns the kn-rendered page.
- **CI key-parity:** guarantees every locale has every key, so a correctly-negotiated locale can never fall through to a raw key.
- **End-to-end (manual, owner machine ? per I18N_PLAN §6):** set the real OS/browser to Kannada, then Hindi ? load both surfaces ? confirm the right language renders **and the Noto fonts display** (the cloud CI container can't render fonts; the owner eyeball is the final gate).
- **? Query parsing / semantic understanding (forward-looking, owner directive 2026-06-21) ? i18n testing is MANDATORY there.** The query bar is **English-grammar-only** today (a documented limit, tracker A9): the user types English queries (`over 10s`, `top 15 longest`) and a localized **sample-query prompt** shows them what to type in *every* locale (kn/hi included). The **moment** the parser gains **semantic understanding** or accepts **Kannada/Hindi query input** (e.g. typing a Kannada/Hindi phrase, or a conversational/NL builder ? I18N_PLAN P4), i18n tests become a **merge gate**: a kn/hi query (typed or spoken) must parse to the **correct selection**, the localized **parse-feedback** ("parsed-pill") must render in the active locale, and the unsupported/empty-result messages must localize. **No query-NL / semantic-parsing work merges without those tests.**

A locale that's reachable by the switcher but **not auto-selected from the browser language is a bug, not a "later".** This matrix is a DoD gate (§9).

7. Deliverables & progress tracker   ? **source of truth**
Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking.

ID	Deliverable	Workstream	Depends on	Gated?	Status	PR
P0	This tracked project doc + inbound pointers	?	?	No	?	[#41](https://github.com/avidullu/khelsutra/pull/41) · engine [#275](https://github.com/Khelsutra/badminton-highlight-indexer/pull/275)
B1	**SPA i18n scaffold + language switcher, en-only** (react-i18next + detector + `en` catalog + switcher + persistence + **jsdom render tests** + **browser-language negotiation tests** §6.5 N1/N3?N7)	B	P0	No	?	[#42](https://github.com/avidullu/khelsutra/pull/42)
B2	Extract SPA strings ? `en` catalog (~50 literals + ~16 derived; ICU plural keys); add **key-parity + no-bare-strings CI**	B	B1	No	?	[#45](https://github.com/avidullu/khelsutra/pull/45)
A1	Plain-language copy pass on the `en` catalog (de-jargon; icon+text; non-color selection)	A	B2	No	?	?
B3	Tiny shared `t()` shim + `data-i18n` plumbing for `site/`, en-only (proves shared catalogs across surfaces)	B	B2	D-CATALOG-HOME	?	?
A2	Localized **WhatsApp share** for the reel (SPA + site)	A	B1/B3	No	?	?
A6	Plain-language framing for Request-Access GPU/storage questions	A	B3	No	?	?
A7	Fix English-source recording gaps (add a placement diagram) before localizing	A	?	No	?	?
B4	**Noto Sans Kannada + Devanagari** bundle (OFL carve-out) ? *SPA done; reel

watermark `font_path` = engine follow-up* | B | ? | D-FONT ? | ? | [#46](https://github.com/avidullu/khelsutra/pull/46) |
138 | B5 | **Kannada (`kn`) catalog** ? render ? *SPA done (AI-draft, native review pending); `site/` rides B3* | B | B2/B4 | D-LOCALES ? | ? | [#47](https://github.com/avidullu/khelsutra/pull/47) |
139 | A3 | Localize **recording-education** (capture + homepage gist) to kn | A | B3/B5 | D-LOCALES+D-FONT | ? | ? |
140 | A4 | Mobile-first touch targets + tooltip-text ? visible (SPA) | A | B1 | No | ? | ? |
141 | A5 | Friendly engine-error mapping over raw FastAPI `detail` (SPA, client-side map) | A | B2 | No | ? | ? |
142 | B6 | **Hindi (`hi`) catalog** ? render (the extensibility proof ? just a catalog) ? *SPA done (AI-draft, native review pending)* | B | B5 | D-LOCALES ? | ? | [#48](https://github.com/avidullu/khelsutra/pull/48) |
143 | A8 | Background-processing reassurance + verdict/status as icon chips (SPA) | A | B2 | No | ? | ? |
144 | A9 | Tap-only filter controls alongside the English query bar (SPA) | A | B2 | No | ? | ? |
145 | B7 | *(deferred, cross-repo)* Engine **P2 backend customer-facing localization** (report `customer_*` + rejection reasons) | B | engine | **D-BACKEND-DEPTH, owner-gated** | ? | ? |
146
147 **? MILESTONE ? SPA fully trilingual (2026-06-21):** the `web/` Studio SPA now speaks **en + kn + hi** end to end (B1?B2?B4?B5?B6, all merged). A **Kannada-default browser auto-renders Kannada**, and the switcher walks kn ? hi ? en ? **proven by a Playwright screencast** with `locale: kn-IN` (`khelsutra-i18n-screencast.mp4`; frames captured per locale + `<html lang>` tracked). This meets the owner's success criterion **on the SPA surface**. **Still open for the full §9 DoD:** **B3** (the `site/` marketing surface via the shared `t()` shim ? the homepage/capture pages are still English-only), **A3** (recording-education in kn/hi), the **non-tech UX A-items**, and a **native-speaker review** of the AI-drafted kn/hi catalogs before a wide public launch. Do **not** archive until both surfaces are trilingual.
148
149 ## 8. Open questions ? owner (blocking gates) `[design]`
150 *Status (2026-06-21): 4 of 5 **LOCKED/ADOPTED** by the owner ? see §2 (D6?D9). Only D-BACKEND-DEPTH remains open (and it's deferred, not a v1 blocker).*
151 1. **D-FONT ? ? LOCKED ? (a) OFL-for-media carve-out, self-host Noto Sans Kannada/Devanagari** (counsel may still confirm). An explicit exception to the MIT/Apache/BSD code guardrail (fonts = media). Unblocks B4/B5/A3 + the kn/hi reel watermark.
152 2. **D-LOCALES ? ? LOCKED ? `en` + `kn` for v1, `hi` next, always-fallback to `en`** (both 2-form CLDR ? `hi` is catalog-only).
153 3. **D-BACKEND-DEPTH ? ? OPEN (deferred, not a v1 blocker).** Recommend **customer verdict/report `customer_*` + validation-rejection reasons first; technical/operator text stays English; phase two** (engine repo, owner-gated ? tracker row B7).
154 4. **D-SURFACE-FIRST ? ? ADOPTED ? lead with the SPA scaffold (B1), localize site recording-education early, SKIP the dead static-UI retrofit.**
155 5. **D-CATALOG-HOME ? ? ADOPTED ? co-locate `locales/` + CI in `khelsutra`** (a shared i18n dir consumed by both the SPA and the site). (The engine `I18N_PLAN` assumed the indexer repo; this is the adaptation.)
156
157 ## 9. Definition of done
158 - [ ] **Automatic language negotiation (§6.5) is green** ? a **Kannada browser gets Kannada and a Hindi browser gets Hindi** (N1/N2); persisted-pref / `?lang` / region-subtag / unsupported-fallback / missing-key-fallback all proven (N3?N7) on the SPA, and the static-site shim honors `Accept-Language` + `?lang` (N8).
159 - [ ] **Manual cross-check (owner machine):** OS/browser set to Kannada and to Hindi each render the respective language on BOTH surfaces, with Noto fonts displaying.
160 - [ ] `en + kn + hi` catalogs live and **rendering** on BOTH `web/` SPA and `site/` marketing (switcher + persistence + `en` fallback; no raw keys shown).
161 - [ ] **Noto Sans Kannada + Devanagari render correctly** ? owner eyeball on a real machine.
162 - [ ] CI **key-parity + no-bare-strings** green; a stub locale flags every gap.
163 - [ ] **Recording-education** (capture + homepage gist) available in `kn` + `hi`.
164 - [ ] **Non-tech UX baseline:** plain-language copy (no raw engine jargon in user-facing strings), mobile-usable touch targets, icon+text, WhatsApp share, honest today-vs-roadmap preserved through translation.
165 - [ ] §7 all rows ? (or explicitly deferred with a reason) ? flip **Status ? DONE** +

completion note ? run engine `DOC_STATUS.md §2` archival checklist ? `git mv` to `docs/archives/`.

10. References

Internal code anchors `[verified]`: `web/package.json`, `web/src/{App.tsx,main.tsx,components/QueryBar.tsx,components/CompileResult.tsx,lib/query.ts}`; `site/public/{index,capture,own-model}.html`, `site/server.js`, `site/src/assets.test.js`. **This repo's docs:** `docs/PER_RALLY_SELECTION.md`, `docs/REQUEST_ACCESS_FORM.md`, `docs/README.md`, `NEXT_STEPS.md`. **Engine design docs (canonical):** `badminton-highlight-indexer/docs/{I18N_PLAN.md,UI_PROPOSAL.md,VIDEO_RECORDING_GUIDELINES.md,PMF_MARKET_RESEARCH.md,PMF_FIELD_SIGNALS.md,KHELSUTRA_PRODUCT_OVERVIEW.md,DOC_STATUS.md}`. **External `[researched]`:** Unicode CLDR plural rules (kn/hi = one/other); react-i18next + i18next-icu; SIL Open Font License 1.1; Google Noto Sans Kannada/Devanagari.

Changelog

- `2026-06-21` ? created. Isolated the UX + Hindi/Kannada work into one tracked project (owner directive). Grounded in a multi-agent research+verification pass (web/+site/ string maps, Indic-l10n prior art, 5 adversarial verdicts). Decisions D1?D5 locked; 5 product/licensing decisions surfaced as open; §7 tracker seeded with B1 as the recommended first slice.
- `2026-06-21` ? owner locked **D6 (en+kn?hi)** + **D7 (OFL Noto carve-out)** and adopted **D8 (SPA-first)** + **D9 (catalogs in khelsutra)**; only D-BACKEND-DEPTH remains open. **B1 greenlit ? ? in progress.**
- `2026-06-21` ? **B1 opened ? [#42](https://github.com/avidullu/khelsutra/pull/42)**: SPA i18n scaffold (react-i18next + ICU + detector) + `en` catalog + language switcher + the SPA's first jsdom render tests. Verified typecheck/79-tests/build green + live in-browser (switcher persists, en-fallback). Doc/pointer PRs opened: khelsutra [#41](https://github.com/avidullu/khelsutra/pull/41), engine [#275](https://github.com/Khelsutra/badminton-highlight-indexer/pull/275).
- `2026-06-21` ? **§6.5 Testing strategy added (owner ask, #41):** a rigorous **automatic-negotiation matrix** (N1?N9) so a Kannada/Hindi **browser** auto-gets the respective version ? proven at the detection level now (B1), at the content level with the catalogs (B5/B6), on the static shim (B3), and by a manual font-render eyeball. Wired into the DoD (§9). B1 (#42) gains the detection tests (N1/N3?N7).
- `2026-06-21` ? **? SPA MILESTONE: fully trilingual (owner directive "take it to completion + screencast").** B2 (#45, full string extraction + key-parity/no-bare-strings CI) ? B4 (#46, self-host OFL Noto) ? B5 (#47, Kannada) ? B6 (#48, Hindi ? the extensibility proof) all merged. 99 web tests green incl. N1/N2 browser-detection + B5/B6 render + key-parity. **Playwright screencast** (`locale: kn-IN`): a Kannada-default browser auto-renders Kannada ? switch hi ? switch en, each with `<html lang>` tracked (`khelsutra-i18n-screencast.mp4` + per-locale frames). kn/hi are **AI-drafted ? native review pending**. **Remaining for full §9 DoD:** B3 (the `site/` marketing surface), A3 + the UX A-items, native-speaker translation review.

16. assistant (2026-06-21T13:52:41.860Z)

I now have deep grounding. The CLI today does NOT open a browser; `--server` just boots FastAPI on `127.0.0.1:8000`. D10 is explicit. Let me do focused web research on the launcher options. Let me load the WebSearch tool.

17. user (2026-06-21T13:52:54.105Z)

Web search results for query: "pywebview vs Tauri sidecar Python FastAPI local server launcher system tray Windows"

Links: [{"title":"GitHub - dieharders/example-tauri-v2-python-server-sidecar: An example desktop app built with Tauri version 2. Bundle a Next.js frontend with a Python api server. · GitHub","url":"https://github.com/dieharders/example-tauri-v2-python-server-sidecar"},{"title":"GitHub - dieharders/example-tauri-python-server-sidecar: An example desktop app built using the Tauri framework to bundle Next.js with a Python server. · GitHub","url":"https://github.com/dieharders/example-tauri-python-server-sidecar"},{"title":"PyTauri Bridges Python and Tauri, Sparking Debate on WebView Desktop Apps - BigGo News","url":"https://biggo.com/news/202510140726_PyTauri_Python_Tauri_Binding"},{"title":"How to write and package desktop apps with Tauri + Vue + Python | by Senhaji Rhazi hamza | Medium","url":"https://hamza-senhajirhazi.medium.com/how-to-write-and-package-desktop-apps-with-tauri-vue-python-ecc08e1e9f2a"},{"title":"Building Production-Ready Desktop LLM Apps: Tauri, FastAPI, and PyInstaller","url":"https://aiechoes.substack.com/p/building-production-ready-desktop"},{"title":"GitHub - pytauri/pytauri: Tauri binding for Python through Pyo3 · GitHub","url":"https://github.com/pytauri/pytauri"},{"title":"PyTauri","url":"https://pytauri.github.io/pytauri/0.8/"}]

I found search results for your query. Here's a summary of the information about pywebview vs Tauri with Python FastAPI sidecars and local server launchers:

Key Findings:

Tauri with Python FastAPI Sidecar Pattern:

A native app built with Tauri version 2 can spawn a Python sub-process (sidecar) which starts a FastAPI server. Tauri's sidecar pattern keeps things clean, with the Python backend and Rust frontend as separate processes communicating over HTTP, making development easier and the architecture more resilient since one component crashing doesn't take down the other.

Tauri wraps the native WebView provided by the operating system (WebKit on macOS/Linux, WebView2 on Windows), resulting in dramatically smaller binary sizes.

PyWebView as an Alternative:

PyTauri (and by extension Tauri) aims to become a viable alternative to existing Python GUI solutions like pywebview and Pystray, offering the ability to use a JS front-end with a Python backend for a local app.

System Tray Support:

Future improvements could include system tray integration, and better error handling for network issues. The search results mention system tray as a potential feature but don't detail current implementations.

Windows-Specific Benefits:

End users just double-click and run with no Python installation, no virtual environments, avoiding "works on my machine" debugging sessions.

The search results show practical examples of Tauri with Python FastAPI on GitHub, but don't contain detailed comparisons of features like system tray implementation between pywebview and Tauri approaches.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

18. user (2026-06-21T13:52:55.526Z)

Web search results for query: "FastAPI uvicorn open browser localhost webbrowser.open desktop app non-technical user Windows"

Links: [{"title":"First Steps - FastAPI","url":"https://fastapi.tiangolo.com/tutorial/first-steps/"},{"title":"FastAPI - Uvicorn - GeeksforGeeks","url":"https://www.geeksforgeeks.org/python/fastapi-uvicorn/"},{"title":"uvicorn url is not working locally. http://127.0.0.1:8000 · fastapi/fastapi · Discussion #10663","url":"https://github.com/fastapi/fastapi/discussions/10663"},{"title":"FastAPI with Uvicorn: A Comprehensive Tutorial | Orchestra","url":"https://www.getorchestra.io/guides/fastapi-with-uvicorn-a-comprehensive-tutorial"},{"title":"FastAPI Tutorial in Visual Studio Code","url":"https://code.visualstudio.com/docs/python/tutorial-fastapi"},{"title":"Running Your First FastAPI Application","url":"https://codesignal.com/learn/courses/introduction-to-fastapi-basics/lessons/running-your-first-fastapi-application"},{"title":"bam bam bhole! how, endpoint

accessing on mobile browser i.e chrome LOCALLY using IP Address and PORT beacuse simple *Python* python - m http.server accessible using having port 8000 accessible on mobile After Setting windows defender Inbound rule And allow app and feature defender setting but How FastAPI to access endpoint on mobile browser · fastapi/fastapi · Discussion #11537","url":"https://github.com/fastapi/fastapi/discussions/11537"}]

Based on the search results, here's what I found related to FastAPI, uvicorn, opening browsers at localhost, and creating desktop apps for non-technical users on Windows:

Key Findings:

Running FastAPI with Uvicorn:
Uvicorn runs FastAPI apps on http://127.0.0.1:8000 by default, and you can access the API documentation by navigating to http://localhost:8000/docs.

Accessing the Application:
127.0.0.1 refers to localhost, which is the default hostname for your own computer, with :8000 signifying the server is listening on port 8000, so you can open a web browser and navigate to http://localhost:8000.

For Windows Users:
In Visual Studio Code, you can Ctrl+Click the http://127.0.0.1:8000/ URL in the terminal to open your default browser to that address.

For Non-Local Access:
The host 0.0.0.0 means the application will listen on all network interfaces, in contrast to the default host 127.0.0.1, which means the application will only be accessible from the local machine.

Programmatic Approach:
For production use with more control, you can run Uvicorn programmatically with uvicorn.run("app:app", host="0.0.0.0", port=8000, log_level="info"), which would allow you to add custom logic like automatically opening the browser using Python's `webbrowser` module for desktop app distribution.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

19. assistant (2026-06-21T13:53:03.232Z)

Let me dig into the production-ready Tauri+FastAPI+PyInstaller article and search for the alternative packaging approaches (PyInstaller, pystray tray, one-click installers like Inno Setup), plus the Open WebUI / Ollama precedent for how mature local-AI apps handle this.

20. user (2026-06-21T13:53:11.804Z)

Desktop LLM App Architecture Summary

Core Architecture

The system combines three layers:

1. **Tauri** (Rust frontend) - lightweight native UI using system webview
2. **FastAPI** (Python backend) - asynchronous API with streaming support via Server-Sent Events
3. **llama.cpp** - CPU/GPU inference engine bundled with PyInstaller

Server Startup Flow

The Rust setup code spawns the Python backend as a sidecar process during app initialization:

> "The sidecar process runs independently. Tauri manages its lifecycle?start it on app launch,

kill it on app close."

The sidecar runs on port 8000, managed by Tauri's shell API with UTF-8 environment variables configured.

Port Conflicts

The article identifies this as the most common runtime issue: "Port conflicts are the most common issue. If port 8000 is already in use, the backend fails to start silently." Resolution requires finding and terminating conflicting processes manually.

Pain Points Identified

Not addressed:
- System tray integration
- Binary signing
- torch/heavy ML dependencies (only llama.cpp mentioned)

Explicitly mentioned challenges:
- Binary size: ~40MB for Python executable + llama.cpp
- Startup overhead: "5-10 seconds with model loading"
- UTF-8 encoding crashes on Windows requiring environmental fixes
- Sidecar naming conventions (target triple format) causing build failures

Missing Sidecar Lifecycle Features

The article notes potential improvements but doesn't implement them: "Future improvements could include model swapping without reinstall, conversation persistence with SQLite, system tray integration, and better error handling."

21. user (2026-06-21T13:53:16.766Z)

Web search results for query: "PyInstaller bundle PyTorch CUDA large binary size problems alternative conda separate environment ML app packaging"

Links: [{"title":"Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552","url":"https://github.com/orgs/pyinstaller/discussions/8552"},{"title":"PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/7175"},{"title":"Error for binaries larger than 2Gb · Issue #3939 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/3939"},{"title":"Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/8551"},{"title":"How to reduce the size of the PyTorch package after using PyInstaller with -D option, resulting in a 3GB size for the torch file? - PyTorch Forums","url":"https://discuss.pytorch.org/t/how-to-reduce-the-size-of-the-pytorch-package-after-using-pyinstaller-with-d-option-resulting-in-a-3gb-size-for-the-torch-file/194822"},{"title":"Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net","url":"https://www.codegenes.net/blog/pyinstaller-pytorch/"},{"title":"Libtorch_cuda.so is too large (>2GB) - deployment - PyTorch Forums","url":"https://discuss.pytorch.org/t/libtorch-cuda-so-is-too-large-2gb/103155"},{"title":"Compatibility with bitsandbytes · Issue #7911 · pyinstaller/pyinstaller","url":"https://github.com/pyinstaller/pyinstaller/issues/7911"}]

Links: [{"title":"PyTorch and Chill: How to Set Up Your Conda Environment and Docker Container for PyTorch | by Trent Leslie | Medium","url":"https://medium.com/@trentleslie/pytorch-and-chill-how-to-set-up-your-conda-environment-and-container-for-pytorch-c7424514fa17"},{"title":"Packaging Python and PyTorch for a Machine Learning Application | Transformer Lab","url":"https://lab.cloud/blog/packaging%20python/"},{"title":"Setting up Conda, TensorFlow, and PyTorch for your Deep Learning Experimentation | by Ehsan Yousefzadeh-Asl-Miandoab | Computing Systems and Hardware for emerging applications (especially Machine Learning, Deep Learning) | Medium","url":"https://medium.com/mlearning-ai/setting-up-conda-tensorflow-and-pytorch-for-your-deep-learning-experimentation-5d6b0e3c1fe0"},{"title":"Multiple anaconda environments for different Pytorch versions - PyTorch

Forums","url":"https://discuss.pytorch.org/t/multiple-anaconda-environments-for-different-pytorch-versions/173900"},{"title":"Mastering Conda Environments, YAML, and PyTorch ? codegenes.net","url":"https://www.codegenes.net/blog/conda-environment-yml-pytorch/"},{"title":"Guide to Conda for TensorFlow and PyTorch | Towards Data Science","url":"https://towardsdatascience.com/guide-to-conda-for-tensorflow-and-pytorch-db69585e32b8/"},{"title":"Install PyTorch | Environment Setup","url":"https://apxml.com/courses/getting-started-with-pytorch/chapter-1-pytorch-fundamentals-setup/installation-environment-configuration"},{"title":"Pytorch with Conda and Cuda - ML Cloud User Guide","url":"https://portal.mlcloud.uni-tuebingen.de/user-guide/tutorials/pytorch_cuda/"},{"title":"Setting up an Environment for Machine Learning with Conda & Pip-Tools | Towards Data Science","url":"https://towardsdatascience.com/setting-up-an-environment-for-machine-learning-with-conda-pip-tools-9e163cb13b92/"}]

Links: [{"title":"How to deploy a pytorch detector built by pyinstaller in ubuntu to another host with cuda of different version - PyTorch Forums","url":"https://discuss.pytorch.org/t/how-to-deploy-a-pytorch-detector-built-by-pyinstaller-in-ubuntu-to-another-host-with-cuda-of-different-version/192842"},{"title":"Pyinstaller with pytorch - PyTorch Forums","url":"https://discuss.pytorch.org/t/pyinstaller-with-pytorch/47682"},{"title":"Pyinstaller & pytorch - deployment - PyTorch Forums","url":"https://discuss.pytorch.org/t/pyinstaller-pytorch/185917"},{"title":"Top PyTorch Alternatives in 2026","url":"https://slashdot.org/software/p/PyTorch/alternatives"},{"title":"Cross-platform PyTorch stand-alone executables - jit - PyTorch Forums","url":"https://discuss.pytorch.org/t/cross-platform-pytorch-stand-alone-executables/49475"},{"title":"GitHub - pytorch/pytorch: Tensors and Dynamic neural networks in Python with strong GPU acceleration · GitHub","url":"https://github.com/pytorch/pytorch"}]

Based on the search results, here's a comprehensive overview of the PyInstaller + PyTorch + CUDA deployment challenges and alternatives:

The Core Problem

Most of PyTorch's size comes from binary extensions and CUDA/cuDNN shared libraries, which cannot be split. This creates a fundamental issue where the original torch2.3+cu118 in conda is around 5GB, and the torch library in the app is about 4GB.

Size Optimization Attempts

If you must use PyInstaller, limited optimization is possible:

- If you are not using CUDA in your application, you can exclude CUDA-related modules using options like --exclude-module=torch.cuda
- You can optimize the package size by removing unnecessary imports from your Python script, only importing the modules and functions that you actually need, and using the --exclude-module option to exclude certain modules that are not required

However, those who create smaller executables likely either use CPU-only torch builds (no CUDA), or some older build with older (and smaller) CUDA libs.

Recommended Alternatives

Conda Environment Approach:
Conda/Miniconda is the most common packaging tool for Python ML applications because Conda installs a specific version of Python for you in an environment, and it makes it easy to install CUDA and CUDA-specific variants of the libraries you need. It is a good practice to create a separate Conda environment for each project, which helps in keeping the dependencies of different projects isolated and avoids conflicts between packages.

Why PyInstaller Falls Short:
PyInstaller is the most common way to compile a Python application into an executable, but if your application is complex and depends on Python packages that are themselves reliant on compiled code, well, good luck getting it all to work.

Docker Containerization:

You can create a Conda environment with the specific version of PyTorch that you need and install all the necessary libraries, and also set up a Docker container with the same environment to ensure that your code would run consistently across different machines.

The consensus is that for complex ML applications with PyTorch and CUDA, conda-based environments (either deployed as-is or in Docker containers) are significantly more practical than attempting to bundle everything with PyInstaller.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

22. assistant (2026-06-21T13:53:21.415Z)

Excellent ? the PyInstaller+torch+CUDA dead-end (3-5GB, CUDA libs can't be split, "good luck") is directly relevant to our hard packaging problem. Let me look at the precedent set by mature local-AI apps (Ollama, Open WebUI, LM Studio, Pinokio) for how they launch + first-run, and the Transformer Lab article which is exactly our shape (Python+PyTorch ML app distribution).

23. user (2026-06-21T13:53:32.090Z)

How Transformer Lab Packages Python + PyTorch

Architecture

Transformer Lab uses a two-part system: an Electron app (React+TypeScript) that communicates with a Python-based API server handling ML tasks.

Packaging Approach

They chose Miniconda/Conda rather than bundling Python directly. The key mechanisms:

- **Separate Installation Script**: An `install.sh` bash script runs at app startup, pulling their git repository and executing sequential installation steps
- **Isolated Conda Environment**: They install Miniconda into a Transformer Lab-specific directory rather than the standard location, ensuring isolation even if users have existing conda installations
- **Pinned Dependencies**: All packages are pinned using `piptools`-generated requirements files
- **Manual Package Installation**: Flash Attention 2 requires separate installation due to compatibility issues with standard pip workflows
- **CUDA Support**: Conda handles CUDA installation when possible

GUI + Server Integration

The Electron app spawns the bash installer and displays real-time CLI output for each step. They deliberately separated steps in the UI so failures are immediately identifiable.

Why Not PyInstaller?

The critical reason: **"Transformer Lab allows users to install Plugins that themselves can install arbitrary Python libraries. So we needed an accessible Python environment that we could modify after it's installed on a user's machine."**

PyInstaller compiles everything into static executables, preventing post-installation customization?a dealbreaker for their plugin architecture.

Key Challenges Solved

They documented dozens of environment-specific quirks: multiple conda versions, admin permission issues, shell activation problems. Their solution involved defensive coding: deactivating conda three times before installation to prevent conflicts.

24. user (2026-06-21T13:53:32.884Z)

Web search results for query: "Ollama Open WebUI LM Studio local AI app first-run setup wizard download model API key non-technical Windows installer"

Links: [{"title":"Part 1: Getting Started with Local AI - Ollama & Open WebUI | by John Wong | Medium","url":"https://medium.com/@able_wong/getting-started-with-local-ai-ollama-open-webui-part-1-bcaafcce6df7"},{"title":"How to Run an LLM Locally: Ollama & Open WebUI Guide","url":"https://www.neurallyintense.com/run-llm-locally/"},{"title":"Setting up a simple Local LLM: Ollama + OpenWebUI with RAG | by @ro0taddict | Medium","url":"https://rodelllemit.medium.com/setting-up-a-simple-local-llm-ollama-openwebui-with-rag-1d78b748c8d4"},{"title":"Local ChatGPT in 10 Minutes: Open WebUI + Ollama | TECHSY","url":"https://techsy.io/en/blog/open-webui-ollama"},{"title":"Connect a Provider / Open WebUI","url":"https://docs.openwebui.com/getting-started/quick-start/connect-a-provider/"},{"title":"Local AI Setup Guide: Ollama, Open WebUI & AnythingLLM (2026) | Silverthread Labs","url":"https://www.silverthreadlabs.com/blog/local-ai-setup-guide"},{"title":"Open WebUI + Ollama on Mac: Local AI for Your Home Network // famstack.dev","url":"https://famstack.dev/guides/open-webui-ollama-mac-setup/"},{"title":"Open WebUI Setup Guide: Local ChatGPT with Ollama (2026) | Local AI Master","url":"https://localaimaster.com/blog/open-webui-setup-guide"},{"title":"A local AI setup using Ollama and Open WebUI","url":"https://www.emcken.dk/programming/2025/07/10/local-ai-setup-using-ollama-and-openwebui/"}]

Based on the web search results, here's what I found about local AI applications including Ollama, Open WebUI, and LM Studio:

Overview

Ollama is a free and open-source tool that simplifies running large language models (LLMs) directly on your local computer by handling the complexities of downloading, setting up, and interacting with various LLMs. Open WebUI is an open-source, self-hosted web interface that gives Ollama a ChatGPT-style chat UI, letting you run private AI models on your own machine with no API keys, no per-token cost, and full data control.

Setup Process

The single-container Docker path gets you to first chat in approximately 10 minutes on a warm machine. The setup involves three steps: install Docker Desktop, install Ollama, then run the Open WebUI container via Docker command, after which you open http://localhost:3000 and create your admin account.

Model Downloads

Once Open WebUI is running, you can pull models directly from its built-in model browser without touching the terminal, which is convenient when you want to try something mid-conversation. When using ollama pull, the default download is typically Q4_K_M, and the recommendation is to start with the default.

Provider Connections

Adding a provider is as simple as entering a URL and API key in Admin Settings ? Connections, and Open WebUI auto-detects available models from most providers.

Alternatives

LM Studio is described as a sleek, all-in-one platform for running LLMs with a user-friendly interface. If you're a solo developer who just wants to run a model on your gaming GPU and chat, LM Studio is faster to set up.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

25. assistant (2026-06-21T13:53:38.344Z)

The Transformer Lab precedent is gold ? it's nearly our exact shape (Electron + Python/PyTorch API server + Miniconda + plugin-installable libs ? rejected PyInstaller because it can't modify the env post-install; the WASB conda env is our parallel). Let me do two more targeted searches: pystray/system tray for "open browser" launchers, and Windows one-click installer options (winget/MSIX/Inno Setup) for non-tech users, plus confirm the localhost binding / WebView2 nuance for India/Windows.

26. user (2026-06-21T13:53:47.281Z)

Web search results for query: "Windows installer non-technical user Inno Setup vs MSIX vs winget bundle python app uv astral standalone python download"

Links: [{"title":"Inno Setup","url":"https://jrsoftware.org/isinfo.php"},{"title":"Easy Windows Installers for Flutter Apps with Inno Bundle | by Hocine Abdellatif Houari | Medium","url":"https://hahouari.medium.com/easy-windows-installers-for-flutter-apps-with-inno-bundle-2ec9b05d2c75"},{"title":"4. Using Python on Windows ? Python 3.14.6 documentation","url":"https://docs.python.org/3/using/windows.html"},{"title":"innosetup · PyPI","url":"https://pypi.org/project/innosetup/"},{"title":"Create Installer of Python Program for Windows OS using Inno ...","url":"https://manaj.hashnode.dev/create-installer-of-python-program-for-windows-os-using-inno-setup"},{"title":"Use WinGet to install and manage applications | Microsoft Learn","url":"https://learn.microsoft.com/en-us/windows/package-manager/winget/"},{"title":"Python Release Python install manager 26.0 | Python.org","url":"https://www.python.org/downloads/release/pymanager-260/"},{"title":"Cyrille Rossant - Create a standalone Windows installer for your Python application","url":"https://cyrille.rossant.net/create-a-standalone-windows-installer-for-your-python-application/"},{"title":"Python Release Python install manager 26.1 | Python.org","url":"https://www.python.org/downloads/release/pymanager-261/"}]

Based on the search results, here's what I found about Windows installer options for Python applications:

Installer Comparison

Inno Setup:
Inno Setup is an open-source installation builder for Windows applications. It supports creation of a single EXE to install your program for easy online distribution and has extensive support for both administrative and non-administrative installations. Inno Setup allows you to customize the installation process as you wish, and together with Python scripts, you can achieve anything.

MSIX:
MSIX is a great pick for those targeting Microsoft Store, for its customization and adhering to Windows Standards. However, MSIX requires signed certificates, which Windows Home edition doesn't allow creating, and the commands to generate them are not friendly.

WinGet:
WinGet is a command line tool enabling users to discover, install, upgrade, remove and configure applications on Windows 10, Windows 11, and Windows Server 2025 computers. WinGet supports multiple installer types including EXE, ZIP, INNO, NULLSOFT, MSI, WIX, APPX, MSIX, BURN, PORTABLE, and FONT.

Python Installation via WinGet

To install Python using WinGet, the command is winget install 9NQ7512CXL7T, and the use of the Store app or the MSIX package is recommended.

Note: The search results didn't contain specific information about UV (Astral's Python package manager) or winget bundle functionality for Python applications. You may need to search specifically for those tools for more detailed guidance.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. user (2026-06-21T13:53:47.485Z)

Web search results for query: "pystray system tray python open browser local server menu quit icon cross-platform license"

Links: [{"title":"Create a Responsive System Tray Icon using Python PyStray - GeeksforGeeks","url":"https://www.geeksforgeeks.org/python/create-a-responsive-system-tray-icon-using-python-pystray/"},{"title":"Reference ? pystray 0.19.5 documentation - Read the Docs","url":"https://pystray.readthedocs.io/en/latest/reference.html"},{"title":"How to terminate a script from the system tray menu · Issue #17 · moses-palmer/pystray","url":"https://github.com/moses-palmer/pystray/issues/17"},{"title":"How to use the pystray.Menu function in pystray | Snyk","url":"https://snyk.io/advisor/python/pystray/functions/pystray.Menu"},{"title":"Tkinter System Tray - Python Tutorial","url":"https://www.pythontutorial.net/tkinter/tkinter-system-tray/"},{"title":"Creating a system tray icon ? pystray 0.19.5 documentation","url":"https://pystray.readthedocs.io/en/latest/usage.html"},{"title":"Creating a Desktop App with System Tray Integration in Python - DEV Community","url":"https://dev.to/techtech/creating-a-desktop-app-with-system-tray-integration-in-python-3ihd"}]

Now let me search for information about pystray's cross-platform support and license:

Links: [{"title":"pystray · PyPI","url":"https://pypi.org/project/pystray/0.3.1/"},{"title":"GitHub - moses-palmer/pystray · GitHub","url":"https://github.com/moses-palmer/pystray"},{"title":"pystray-freedesktop · PyPI","url":"https://pypi.org/project/pystray-freedesktop/"},{"title":"pystray._base ? pystray 0.19.5 documentation","url":"https://pystray.readthedocs.io/en/latest/_modules/pystray/_base.html"},{"title":"Debian -- Details of package python3-pystray in sid","url":"https://packages.debian.org/sid/python3-pystray"},{"title":"pystray 0.19.5 on PyPI - Libraries.io - security & maintenance data for open source software","url":"https://libraries.io/pypi/pystray"},{"title":"pystray._base ? pystray 0.10 documentation","url":"https://pythonhosted.org/pystray/_modules/pystray/_base.html"},{"title":"Reference ? pystray 0.19.5 documentation - Read the Docs","url":"https://pystray.readthedocs.io/en/latest/reference.html"},{"title":"tray-manager · PyPI","url":"https://pypi.org/project/tray-manager/"},{"title":"psgtray · PyPI","url":"https://pypi.org/project/psgtray/"}]

Based on the web search results, here's comprehensive information about pystray:

Overview

PyStray is a module that enables you to make System Tray icons and make them responsive. It enables the specification of an icon, a title, and a callback function for use when the icon is enabled.

Core Features

System Tray Icon Creation:
Pystray provides a representation of a system tray icon through the `pystray.Icon` class. The icon enters a loop handling events, and this method is blocking until `stop()` is called.

Menu Support:
In order to receive notifications about user interaction with the icon, a popup menu can be added with the menu constructor argument, which must be an instance of `pystray.Menu`. The menu will be displayed when the right-hand button has been pressed on the icon on Windows, and when the icon has been clicked on other platforms.

Quit/Exit Functionality

The "Quit" option in the system tray context menu allows users to gracefully exit the application. This action is facilitated by stopping the system tray icon and then destroying the main application window.

Cross-Platform Support

Supported platforms are Linux under Xorg, GNOME and Ubuntu, macOS and Windows. The default backend when running on macOS is available with all pystray features supported, and the default backend when running on Windows is available with all pystray features supported.

License

Pystray is licensed under LGPL-3.0.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. assistant (2026-06-21T13:53:54.903Z)

I have enough for a strong, grounded recommendation. One note: pystray is LGPL-3.0 ? relevant to the project's MIT/Apache/BSD guardrail (LGPL dynamic-link is generally OK but worth flagging). Let me do one final check on the WebView2 dependency for Tauri on Windows (a real first-run friction point for non-tech users in India) and confirm whether Tauri/pywebview need a runtime.

29. user (2026-06-21T13:54:01.704Z)

Web search results for query: "Tauri Windows WebView2 runtime dependency preinstalled Windows 11 vs Windows 10 evergreen bootstrapper pywebview edgechromium"

Links: [{"title":"Webview Versions | Tauri","url":"https://v2.tauri.app/reference/webview-versions/"},{"title":"Windows Installer - The Tauri Documentation WIP","url":"https://jonaskruckenberg.github.io/tauri-docs-wip/building/windows-installer.html"},{"title":"Windows Installer | Tauri","url":"https://v2.tauri.app/distribute/windows-installer/"},{"title":"Prerequisites | Tauri v1","url":"https://v1.tauri.app/v1/guides/getting-started/prerequisites/"},{"title":"Windows Installer | Tauri v1","url":"https://v1.tauri.app/v1/guides/building/windows/"},{"title":"Webview Versions | Tauri v1","url":"https://v1.tauri.app/v1/references/webview-versions/"},{"title":"tauri-docs/src/content/docs/reference/webview-versions.md at v2 · tauri-apps/tauri-docs","url":"https://github.com/tauri-apps/tauri-docs/blob/v2/src/content/docs/reference/webview-versions.md"},{"title":"Prerequisites | Tauri","url":"https://v2.tauri.app/start/prerequisites/"},{"title":"Creating webviews doesn't account for fixed webview2 runtime in Windows environments without any runtime installed · Issue #13817 · tauri-apps/tauri","url":"https://github.com/tauri-apps/tauri/issues/13817"},{"title":"Tauri Tutorial: Build a Cross-Platform App in 13 Steps [2026]","url":"https://tech-insider.org/tauri-tutorial-cross-platform-rust-app-2026/"}]

Based on the web search results, here's what I found regarding Tauri's WebView2 runtime dependency differences between Windows versions:

Windows 11 vs Windows 10: Key Differences

WebView2 comes preinstalled on Windows 11, whereas on versions older than Windows 11 the installer generated by Tauri takes care of ensuring WebView2 is installed on the system.

More specifically, on Windows 10 (April 2018 release or later) and Windows 11, the WebView2 runtime is distributed as part of the operating system.

Installer Behavior

By default, the Tauri installer downloads and installs the Webview2 Runtime if it is not already installed (On Windows 11, WebView2 is preinstalled).

Distribution Methods

For managing WebView2 distribution, you have several options:

1. **Evergreen Bootstrapper**: Download the "Evergreen Bootstrapper" and install it for

automatic updates
2. **Fixed Runtime Version**: Distributing a fixed WebView2 Runtime version increases the Windows Installer by around 180MB
3. **Skip Installation**: You can remove the WebView2 Runtime download check from the installer by setting webviewInstallMode to skip. Your application WILL NOT work if the user does not have the runtime installed and won't attempt to install it

Tauri uses WebView2 which is based on Microsoft Edge and therefore Chromium.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. assistant (2026-06-21T13:54:09.668Z)

I now have everything I need. Let me synthesize. Key grounding established:

- **D10 (verified)**: `LOCAL_APPLIANCE_ARCHITECTURE.md:39` ? delivery = "a **self-hosted webserver + UI the user spins up** (NOT a desktop app)"; explicitly **supersedes `DESKTOP_APP_PLAN.md`**.
- **Today's launch**: `backend/main.py:556,562` ? `main()` with `--server` flag; no `webbrowser`, no tray, no auto-open (grep confirmed). Server binds `127.0.0.1:8000` (`main.py:638` per appliance doc).
- **Hard packaging problem**: WASB needs torch 1.11+cu113 conda env separate from main torch 2.6+cu124; ffmpeg is a system dep; weights in GCS. Web research confirms PyInstaller+torch+CUDA is a 3-5GB dead-end and **Transformer Lab rejected PyInstaller for exactly our reason** (post-install env modification / plugin libs).
- **License**: pystray = LGPL-3.0 (flag vs MIT/Apache/BSD guardrail); Tauri/pywebview/Inno Setup are clean.

31. user (2026-06-21T13:55:56.091Z)

Structured output provided successfully

32. assistant (2026-06-21T13:56:09.227Z)

I've completed the research and returned the RESEARCH schema via StructuredOutput.

Key findings, grounded in code/docs:

- **D10 verdict**: A friendly *launcher* satisfies D10's intent; an embedded-webview shell violates it. D10 (`khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39`) locks "a self-hosted webserver + UI the user spins up (NOT a desktop app)" and explicitly supersedes `DESKTOP_APP_PLAN.md`. A system-tray launcher keeps the rendered product as the khelsutra SPA in the user's own browser at `127.0.0.1:8000` ? it's just a non-scary on/off switch. Option (c) Electron/Tauri IS a desktop app in D10's sense and is rejected.

- **Today's reality**: `backend/main.py:556,562` ? `main()` + `--server`; grep confirms no `webbrowser`, no tray, no auto-open. Server hard-bound to `127.0.0.1:8000`; `GET /api/health` exists for readiness polling.

- **Recommendation**: (b) tiny pystray tray launcher (owns server lifecycle, auto-opens browser, free-port fallback for the documented port-8000 failure, Quit/Restart/Language) + (d) one-click Inno Setup installer that provisions a managed Miniconda/uv env the Transformer-Lab way. Sequenced smallest-first: stdlib `webbrowser.open` after health-poll ? tray wrap ? installer.

- **Hard-packaging fit**: PyInstaller+torch+CUDA is a 3-5GB "CUDA libs can't be split" dead-end, and Transformer Lab rejected PyInstaller for exactly our reason (need a post-install-modifiable env ? our separate WASB torch-1.11 conda env is the parallel). The installer, not a frozen bundle, resolves the `--index-url` torch wheels + GCS weights + system ffmpeg.

- **First-run wizard**: in the web UI (Open WebUI/Ollama precedent) ? (1) Gemini key with consent copy, (2) compute pick rendered from `GET /api/capabilities`/PROFILE_PRESETS, (3) videos folder ? kept en/kn/hi to match the trilingual SPA.

- **License flag**: pystray is LGPL-3.0 (not in the MIT/Apache/BSD guardrail); resolve dynamic-link sign-off or swap to an MIT tray lib before shipping (b). Inno Setup, Tauri, stdlib `webbrowser` are clean.

Relevant files: `C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`, `C:/Users/avidu/Projects/badminton-highlight-indexer/docs/DESKTOP_APP_PLAN.md`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py`, `C:/Users/avidu/Projects/khelsutra/docs/FRIENDLY_AND_MULTILINGUAL.md`.